

CS-Mamba: Grid-Parallel Gated Spatial Recurrence for Vision

1 Problem Statement

Images are naturally defined on a two-dimensional spatial grid, but many efficient sequence models operate on one-dimensional token sequences. When a patchified image is flattened into a sequence, the model must impose an artificial traversal order over the grid. Directional scan methods reduce this issue by scanning the image in multiple directions, but the resulting computation is still sequence-like: information is propagated along ordered paths rather than through the native two-dimensional neighborhood structure of the image.

This creates a mismatch between the structure of the data and the structure of the computation. Local image evidence is inherently geometric: edges, textures, contours, and object parts are formed by interactions among nearby patches in the horizontal, vertical, and diagonal directions. A vision recurrence should therefore preserve the image grid and update all spatial locations in parallel, while still allowing information to propagate beyond immediate neighbors through repeated computation.

The problem addressed in this work is:

Can we build a Mamba-style vision block that performs recurrent spatial reasoning directly on the 2D patch grid, without relying on directional sequence scans, while preserving linear-time scaling in the number of patches?

2 Novelty

CS-Mamba proposes a grid-parallel spatial recurrence for visual recognition. The model keeps the patch representation as a three-dimensional token tensor

$$x \in \mathbb{R}^{B \times N \times D},$$

where B is batch size, $N = HW$ is the number of image patches, and D is the expanded channel dimension. During the recurrent update, this tensor is reshaped into the image grid:

$$\mathbb{R}^{B \times N \times D} \longleftrightarrow \mathbb{R}^{B \times D \times H \times W}.$$

Thus the model remains compatible with standard token projections while performing the core recurrence over the native two-dimensional patch field.

The novelty consists of four components.

Grid-parallel recurrence. All patch locations are updated simultaneously at each recurrence step. The model does not scan through tokens sequentially. Instead, spatial context propagates through local neighborhood interaction over the patch grid.

Gated local spatial evolution. CS-Mamba combines Mamba-style selective gates with a fixed local spatial operator. The gates determine channel-wise self dynamics, input injection, and diffusion strength, while the spatial operator propagates information through neighboring patches.

Parameter-free recurrence depth. The number of recurrence steps K controls computational depth and effective receptive field without changing the number of parameters. Increasing K allows information to travel farther across the image grid, trading compute for accuracy.

Optional nonlinear recurrent dynamics. The default recurrence is a gated linear spatial evolution, which is fast and stable. We also support nonlinear recurrence by applying an activation function inside each recurrent step. This converts the block into a weight-shared nonlinear dynamical system without adding parameters.

3 Proposed Solution

Given an input image $I \in \mathbb{R}^{B \times 3 \times R \times R}$, a patch embedding layer maps the image into a token grid:

$$x_0 \in \mathbb{R}^{B \times N \times d}, \quad N = HW = \left(\frac{R}{P}\right)^2,$$

where P is patch size. A CS-Mamba block first normalizes and expands the token features:

$$[u, z] = W_{\text{in}} \text{LN}(x), \tag{1}$$

$$u = \text{SiLU}(\text{DWConv}_{3 \times 3}(u)). \tag{2}$$

Here u is the branch passed to the spatial recurrence and z is an output gate. The depthwise convolution is a lightweight local preprocessing step before recurrent spatial propagation.

The core recurrence evolves a hidden state $h_k \in \mathbb{R}^{B \times N \times D}$ for K steps. The hidden state is initialized using a bounded input gate:

$$h_0 = u \odot \tanh(W_B u).$$

At each step, the model combines three forces:

1. a stable self term that preserves channel-wise recurrent memory,
2. an input injection term that anchors the dynamics to the current block input,
3. a local spatial diffusion term that exchanges information among neighboring patches.

The final recurrent state is modulated by a sigmoid output gate:

$$\tilde{y} = h_K \odot \text{sigmoid}(W_C u),$$

and the block output is

$$y = x + W_{\text{out}}(\tilde{y} \odot \text{SiLU}(z)).$$

4 Methodology

4.1 Selective Gates and Stable Dynamics

Let $u \in \mathbb{R}^{B \times N \times D}$ be the expanded feature tensor entering the spatial recurrence. CS-Mamba constructs two input-dependent positive gates:

$$\Delta_{\text{self}}(u) = \text{clip}(\text{softplus}(W_{\text{self}}u), 0, \Delta_{\text{max}}), \quad (3)$$

$$\Delta_{\text{diff}}(u) = \text{clip}(\text{softplus}(W_{\text{diff}}u), 0, \Delta_{\text{max}}). \quad (4)$$

The first gate controls self dynamics and input injection; the second controls spatial diffusion. The clipping constant Δ_{max} stabilizes the explicit Euler update by preventing large per-step changes.

The channel-wise self dynamics are parameterized as negative values:

$$A = -\text{softplus}(A_{\log}),$$

which biases the recurrent system toward stable decay rather than uncontrolled growth. The diffusion coefficient is bounded as

$$D_{\text{phys}} = 0.5 \cdot \text{sigmoid}(\theta_D).$$

This keeps the spatial update numerically controlled while allowing each channel to learn a different diffusion strength.

4.2 Four-Nighbor Spatial Recurrence

The default CS-Mamba spatial operator is a four-neighbor Laplacian on the patch grid. Before applying the operator, the hidden state is reshaped:

$$h_k \in \mathbb{R}^{B \times N \times D} \rightarrow h_k^{2D} \in \mathbb{R}^{B \times D \times H \times W}.$$

For a grid location (i, j) , the four-neighbor Laplacian is

$$\mathcal{L}_4(h)_{i,j} = h_{i-1,j} + h_{i+1,j} + h_{i,j-1} + h_{i,j+1} - 4h_{i,j}.$$

Boundary values use replication padding, corresponding to a discrete Neumann boundary condition.

With $dt = 1/K$, define

$$c(u) = 1 + dt \cdot \Delta_{\text{self}}(u) \odot A, \quad (5)$$

$$b(u) = dt \cdot \Delta_{\text{self}}(u) \odot h_0, \quad (6)$$

$$d(u) = dt \cdot \Delta_{\text{diff}}(u) \odot D_{\text{phys}}. \quad (7)$$

The recurrence update is

$$h_{k+1} = c(u) \odot h_k + b(u) + d(u) \odot \mathcal{L}_4(h_k).$$

This update is performed for $k = 0, \dots, K - 1$. All spatial locations are updated in parallel within each step. After K steps, information can propagate across approximately K local grid hops.

4.3 Nonlinear Recurrent Evolution

The default recurrence uses the identity activation inside each step:

$$h_{k+1} = g(h_k, u),$$

where g denotes the gated spatial update above. To increase expressivity without adding parameters, we also evaluate

$$h_{k+1} = \phi(g(h_k, u)),$$

where

$$\phi \in \{\text{identity}, \text{SiLU}, \text{tanh}, \text{GELU}, \text{ReLU6}, \text{ReLU}\}.$$

With $\phi = \text{identity}$, the recurrent evolution is linear in h_k for fixed gates, while the gates remain nonlinear functions of the input. With a nonlinear ϕ , the recurrence becomes a weight-shared nonlinear dynamical system:

$$h_K = \phi(g(\phi(g(\cdots \phi(g(h_0, u))))), u).$$

This increases computational expressivity without increasing parameter count. In practice, smooth activations such as SiLU are especially attractive because they preserve negative state values and are less brittle than hard rectification.

4.4 Eight-Nighbor Laplacian Ablation

The main model uses the four axial neighbors. We also study an eight-neighbor variant that includes diagonal interactions:

$$\begin{aligned} \mathcal{L}_8(h)_{i,j} = & h_{i-1,j} + h_{i+1,j} + h_{i,j-1} + h_{i,j+1} \\ & + h_{i-1,j-1} + h_{i-1,j+1} + h_{i+1,j-1} + h_{i+1,j+1} - 8h_{i,j}. \end{aligned}$$

This ablation tests whether diagonal propagation improves accuracy. It is expected to be slightly more expensive because it reads eight neighbors instead of four. If the four-neighbor operator matches the eight-neighbor operator, the simpler operator is preferred for efficiency. If the eight-neighbor operator improves accuracy, it represents a clear accuracy-speed tradeoff.

4.5 Learned Local Operator Ablations

To test whether the fixed Laplacian is essential, we compare it with learned depthwise local operators. A depthwise 2D convolution replaces $\mathcal{L}_4$ with a learned 3×3 kernel initialized as

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

We also evaluate a depthwise 1D convolution over the flattened grid, initialized as the second-difference stencil

$$[1, -2, 1].$$

These ablations separate the contribution of the proposed recurrent formulation from the exact choice of spatial operator. If learned Conv1D or Conv2D operators obtain similar accuracy but slower execution, the fixed Laplacian is justified as an efficient spatial prior.

4.6 Complexity and Receptive Field

Let $N = HW$ be the number of patches, D the expanded channel dimension, and K the recurrence depth. The four-neighbor recurrence has time complexity

$$\mathcal{O}(BKND)$$

and stores a recurrent state of size

$$\mathcal{O}(BND).$$

The number of learned parameters is independent of K . Thus K acts as a parameter-free expressivity knob:

$$K \uparrow \Rightarrow \text{larger effective receptive field and deeper computation.}$$

For a token grid of size $T = N$, a natural large-context setting is

$$K = \lceil \log_2 T \rceil.$$

For Tiny-ImageNet with 64×64 images and 4×4 patches, $T = 256$ and $K = 8$. For ImageNet-1K with 224×224 images and 16×16 patches, $T = 196$ and $K = 8$. Smaller values such as $K = 3$ provide a faster local-recurrence setting and are useful for accuracy-efficiency tradeoff studies.

5 Empirical Questions

The method directly motivates the following empirical questions.

Does grid-parallel recurrence outperform directional scans? The primary comparison is against scan-based vision SSM baselines under matched training settings. The desired outcome is equal or better accuracy with lower epoch time or higher throughput.

How much recurrence depth is needed? Ablating K measures whether additional recurrent computation improves accuracy without increasing model size.

Is the fixed Laplacian sufficient? Comparing four-neighbor Laplacian, eight-neighbor Laplacian, learned Conv2D, and learned Conv1D tests whether the model depends on a specific hand-crafted stencil or whether local recurrent spatial propagation is the main source of performance.

Does nonlinear recurrence help? Comparing identity, SiLU, tanh, GELU, ReLU6, and ReLU inside the recurrent update tests whether parameter-free nonlinear recurrent computation improves visual recognition while preserving the grid-parallel execution pattern.